Subject Code: KCA204

Roll No. :_____

MCA
(SEM II) THEORY EXAMINATION 2023-24
DATABASE MANAGEMENT SYSTEMS

Time: 3 Hours ***Total Marks: 100***

Note: Attempt all Sections. If require any missing data; then choose suitably.

SECTION A

Note: - Attempt *all* questions in brief. **2 x 10 = 20**

a) What do you mean by logical data independence?

Logical data independence refers to the ability of a database system to change its **logical schema** without requiring any modification in the **external schema (user views)** or application programs. The logical schema defines the structure of the database such as tables, attributes, and relationships.

This means that changes like adding a new attribute, modifying a table structure, or splitting a relation should not affect how users interact with the database.

Example:

If a new column "Salary" is added to the Employee table, existing applications that use only Employee ID and Name should continue to work without any changes.

---

b. What do you mean by data integrity?

Data integrity refers to the **accuracy, consistency, and reliability of data stored in a database**. It ensures that the data remains correct and valid throughout its lifecycle and is not corrupted due to unauthorized access, insertion of invalid data, or system errors.

To maintain data integrity, DBMS uses constraints such as:

- Primary key constraint (ensures uniqueness)
- Foreign key constraint (ensures referential integrity)
- NOT NULL constraint (prevents empty values)
- Check constraint (ensures valid values)

Example:

An employee ID must always be unique, and age cannot be negative. These rules ensure that the database remains meaningful and reliable.

---

c. Define entity and entity set. Give one example for illustration.

Entity:

An entity is a **real-world object or thing that has a distinct existence** and can be identified uniquely. It can be a person, place, object, or event.

Example:

An employee with details like:

- EID = 101
- Name = Rahul

represents a single entity.

Entity Set:

An entity set is a **collection of similar types of entities** that share the same attributes.

Example:

All employees working in an organization form the Employee entity set.

---

d. Explain with suitable example: Stored & Derive Attribute.

Stored Attribute:

A stored attribute is an attribute whose value is **physically stored in the database**. It is directly entered and saved in tables.

Example:
Date of Birth (DOB), Name, and ID are stored attributes because they are explicitly stored.
Derived Attribute:
A derived attribute is an attribute whose value is **calculated or derived from other stored attributes**. It is not stored permanently in the database.
Example:
Age is a derived attribute because it is calculated using Date of Birth and the current date.

---

e. Determine the Relational Algebra expression for following-For a relation Employee(EID, Ename, city), find the name of employees who either live in Delhi or live in Mumbai.

$\pi_{\text{Ename}}(\sigma_{\text{city='Delhi' OR city='Mumbai'}}(\text{Employee}))$

---

f. There is a table named EMP (ID, Name, DOJ). Write the sql query to display all the customer details. (Note:Date of joining must be displayed in the format '07/December/2019')

Ans: SELECT ID, Name, DATE_FORMAT(DOJ, '%d/%M/%Y') AS DOJ FROM EMP;

---

g. What will be the number of possible superkeys for the relation schema R(E,F.G.H) with EF as key?
Given relation: R(E, F, G, H)
Candidate key: EF
A superkey is any set of attributes that contains a candidate key.
Since EF is mandatory in every superkey.
Superkeys:

- {E, F}
- {E, F, G}
- {E, F, H}
- {E, F, G, H}

Total superkeys = **4**

---

h. Consider a relation R (A, B, C, D, E) with following FD's:
A→ BC, CD→ E, B→ D, E → A
Does B→ CD Exists over the relation R?
Ans: Given:

- A → BC
- CD → E
- B → D
- E → A

We need to check whether **B → CD** holds.
Step 1: Find closure of B
$B^+ = \{B\}$
Using B → D:
$B^+ = \{B, D\}$
Now check other dependencies:

- A → BC → not applicable (A not present)
- CD → E → requires C (not present)
- E → A → not applicable

So closure remains:
$B^+ = \{B, D\}$
Conclusion:
Since C is not included in B^+,
B → CD does NOT hold.

---

i. What do you mean by commit point of a transaction?
A commit point is the **exact moment in a transaction when all its operations are successfully completed and the changes are permanently saved in the database**.
Once a transaction reaches the commit point:

- All updates become permanent
- The system guarantees durability
- Rollback is no longer possible

Example:
In a banking system, once a fund transfer is committed, the changes cannot be undone even if the system crashes later.

---

j. What do you mean by a serializable schedule?
A serializable schedule is a schedule in which concurrent execution of transactions produces a result that is **equivalent to some serial execution of those transactions**.
Even though multiple transactions run simultaneously, the final database state remains consistent as if the transactions were executed one after another in some order.
Importance:
It ensures correctness and consistency in concurrent transaction processing.
Example:
If T1 and T2 execute together but produce the same result as T1 → T2 or T2 → T1, then the schedule is serializable.

SECTION B

2a. Explain the advantages of database systems over file-oriented system.
A Database Management System (DBMS) is designed to overcome the limitations of the traditional file-based system. In file-oriented systems, each application maintains its own files, leading to several problems such as redundancy, inconsistency, and difficulty in data sharing. DBMS provides a centralized and efficient way of managing data.
Advantages of DBMS:
1. Reduction in Data Redundancy
In file systems, the same data is stored in multiple files, leading to duplication. DBMS minimizes redundancy by storing data in a centralized database.
2. Data Consistency
Since redundancy is reduced, chances of inconsistency are also reduced. Any update in data is reflected throughout the database.
3. Data Sharing
DBMS allows multiple users and applications to access and share data simultaneously in a controlled manner.
4. Improved Data Security
DBMS provides authentication and authorization mechanisms to restrict unauthorized access to sensitive data.
5. Data Integrity
Integrity constraints (primary key, foreign key, check constraints) ensure accuracy and validity of data.
6. Data Independence
Changes in physical or logical structure of database do not affect application programs.
7. Concurrency Control
DBMS manages simultaneous access of data by multiple users without conflicts or inconsistency.
8. Backup and Recovery
DBMS provides automatic backup and recovery mechanisms to restore data after system failures.
9. Efficient Query Processing
DBMS uses query optimization techniques to retrieve data quickly and efficiently.

10. Reduced Application Development Time
Since DBMS handles data management tasks, developers can focus more on application logic.

2b. Consider the following scenario for a college database. 1. Students enroll for a course. 2. There are some departments. 3. Departments have faculty members. 4. Each department runs some courses. Draw an E-R diagram. Convert the E-R Diagram into relational model.

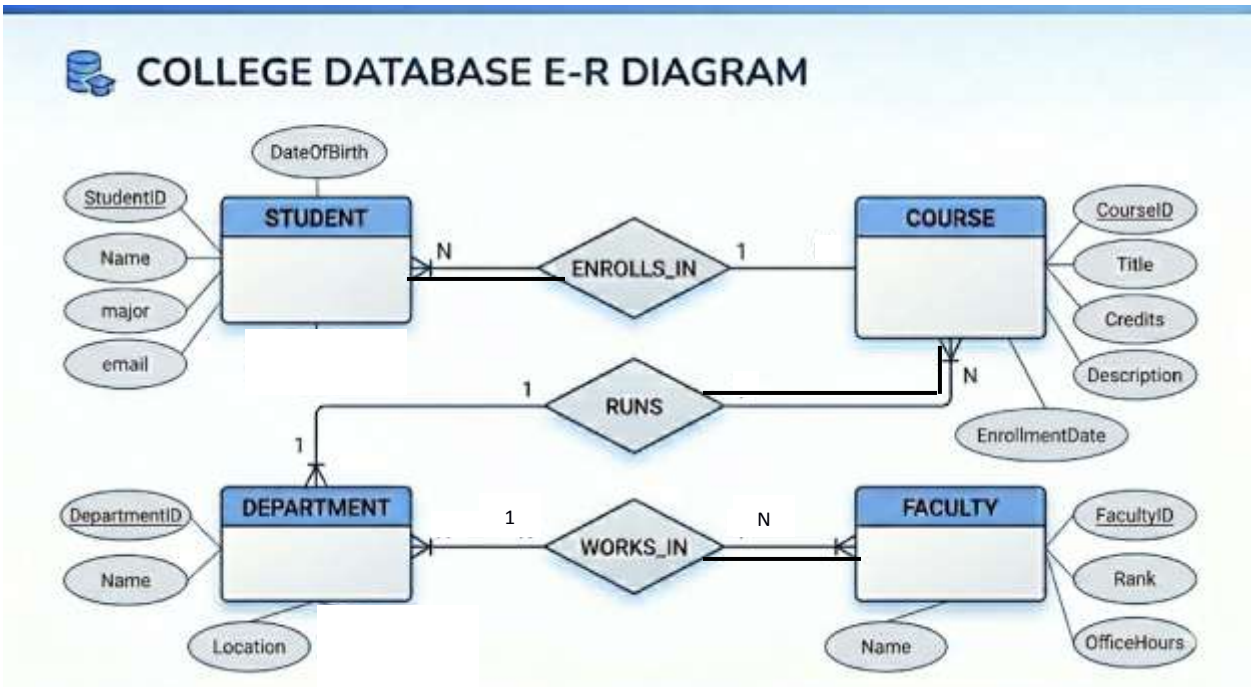
Step 1: Identify Entities and Attributes
Entities:

- Student (Student_ID, Name)
- Department (Dept_ID, Dept_Name)
- Faculty (Faculty_ID, Name)
- Course (Course_ID, Course_Name)

Step 2: Identify Relationships

- Student **Enrolls** Course (Many-to-Many)
- Department **Has** Faculty (One-to-Many)
- Department **Offers** Course (One-to-Many)

Step 3: E-R Diagram Representation (Textual)



Step 4: Conversion into Relational Model
1. Student Table
Student(Student_ID, Name,major,email,dob,courseID)
2. Department Table
Department(Dept_ID, Dept_Name,location)
3. Faculty Table
Faculty(Faculty_ID, Name, Rank,OfficeHours,Dept_ID)
4. Course Table
Course(Course_ID, Title,Credits,Description,EDate, Dept_ID)

2c. What do you mean by Time Stamp? Discuss time Stamp ordering protocol in detail.
Timestamp:

A timestamp is a unique identifier assigned to each transaction based on its start time. It determines the order in which transactions are executed in a DBMS.

With each transaction T_i in the system, we associate a unique fixed timestamp, denoted by TS(T_i). This timestamp is assigned by the database system before the transaction T_i starts execution. If a transaction T_i has been assigned timestamp TS(T_i), and a new transaction T_j enters the system, then TS(T_i) < TS(T_j). There are two simple methods for implementing this scheme:

- Use the value of the **system clock** as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.
- Use a **logical counter** that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

Each data item *Q is associated* two timestamp values:

- **W-timestamp**(*Q*) denotes the largest timestamp of any transaction that executed write(*Q*) successfully.
- **R-timestamp**(*Q*) denotes the largest timestamp of any transaction that executed read(*Q*) successfully.

These timestamps are updated whenever a new read(*Q*) or write(*Q*) instruction is executed.

The Timestamp-Ordering Protocol

The **timestamp-ordering protocol** ensures that any read and write operations are executed in timestamp order. This protocol operates as follows:

Suppose that transaction T_i issues read(*Q*).

a. If TS(T_i) < W-timestamp(*Q*), then T_i needs to read a value of *Q* that was already overwritten. Hence, the read operation is rejected, and T_i is rolled back.

b. If TS(T_i) ≥ W-timestamp(*Q*), then the read operation is executed, and R- timestamp(*Q*) is set to the maximum of R-timestamp(*Q*) and TS(T_i).

Suppose that transaction T_i issues write(*Q*).

c. If TS(T_i) < R-timestamp(*Q*), then the value of *Q* that T_i is producing was needed previously, and the system assumed that the value would never be produced. Hence, the system rejects the write operation and rolls T_i back.

d. If TS(T_i) < W-timestamp(*Q*), then T_i is attempting to write an obsolete value of *Q*. Hence, the system rejects this write operation and rolls T_i back.

e. Otherwise, the system executes the write operation and sets W-time- stamp(*Q*) to TS(T_i).

The timestamp-ordering protocol ensures conflict Serializability. This is because conflicting operations are processed in timestamp order. The protocol ensures freedom from deadlock, since no transaction ever waits. However, there is a possibility of starvation of long transactions if a sequence of conflicting short transactions causes repeated restarting of the long transaction.

2d. Consider a relation R (A, B, C, G, H, I) and set of functional dependencies. F {A→ B, A→ C, CG→ H, CG→ I, B→ H). Is AG a super key for the relation R? Is AG a candidate key?

R(A, B, C, G, H, I)

Functional Dependencies:

- A → B
- A → C
- CG → H
- CG → I
- B → H

Step 1: Find Closure of AG
Start:
$AG^+ = \{A, G\}$
Using $A \rightarrow B$:
$AG^+ = \{A, B, G\}$
Using $A \rightarrow C$:
$AG^+ = \{A, B, C, G\}$
Now using $CG \rightarrow H$:
Since C and G are present → add H
$AG^+ = \{A, B, C, G, H\}$
Using $CG \rightarrow I$:
Add I
$AG^+ = \{A, B, C, G, H, I\}$
Step 2: Check Super Key
AG^+ contains all attributes of R
✔ Therefore, AG is a **super key**
Step 3: Check Candidate Key (Minimality)
Remove A:
$G^+ = \{G\}$ □ not all attributes
Remove G:
$A^+ = \{A, B, C, H\}$ □ missing G, I
So AG is minimal
✔ Therefore, AG is a **candidate key**

2e. Write the reasons for a transaction failure. Explain the check point mechanism used to recover from a transaction failure.
In database management systems, a transaction is a logical unit of processing that must be completed in its entirety to ensure data integrity. When this process is interrupted, it results in a transaction failure.
Reasons for Transaction Failure
Transaction failures can be categorized into three primary types based on where the error originates:
1. Logical or System Errors
- **Internal Logical Errors:** The transaction cannot continue due to an internal condition, such as an arithmetic overflow, division by zero, or an attempt to access data that does not exist.
- **Resource Exhaustion:** The system may lack the necessary resources (like memory, disk space, or CPU cycles) to complete the transaction.

2. System Crashes (Hardware or Software)
- **Power Outages:** A sudden loss of power causes volatile memory (RAM) to clear, losing all active transaction data.
- **Operating System Failure:** A "kernel panic" or software bug in the OS forces the database server to shut down unexpectedly.
- **Hardware Malfunction:** Issues with the CPU or memory controllers can lead to a system-wide halt.

3. Disk or Media Failures
- **Head Crashes:** Physical damage to the hard drive storage or corruption of the disk blocks makes the database files unreadable. This is generally the most severe type of failure, requiring backups for recovery.

The Checkpoint Mechanism
A **Checkpoint** is a mechanism where the system periodically stops and synchronizes all the information in the database log with the actual database files on the disk. It is used to limit the amount of work the system has to do during recovery after a crash.
How Checkpoints Work
Without checkpoints, the system would have to scan the entire log from the beginning of time to recover. With checkpoints, the process is streamlined:

1. **Suspension:** The system temporarily stops the execution of transactions.
2. **Force-Writing:** All log records and modified data buffers (dirty blocks) currently in the main memory (RAM) are written to the stable storage (Disk).
3. **Logging:** A special **[checkpoint]** record is written to the log file, noting all transactions that are currently active.
4. **Resumption:** The system resumes normal transaction processing.

Recovery Using Checkpoints

When a system crash occurs, the recovery manager looks at the log to find the last checkpoint. It then categorizes transactions into two lists:

- **The Undo List:** Transactions that were active at the time of the crash or started after the last checkpoint but never finished (no "Commit" record found). The system reverses their changes to ensure consistency.
- **The Redo List:** Transactions that committed after the last checkpoint. The system reapplies these changes to ensure they are permanently recorded on the disk.

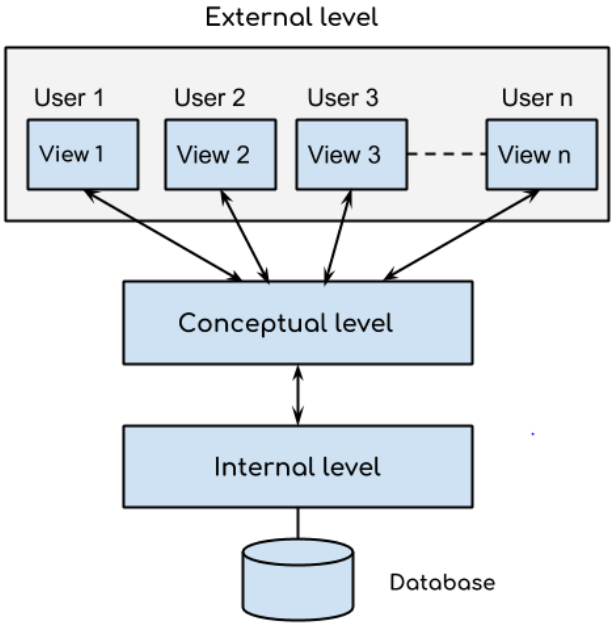
Checkpoints drastically reduce recovery time because the system ignores all transactions that were committed *before* the checkpoint, as their changes are already guaranteed to be safe on the physical disk.

SECTION C

Note: - Attempt any o*ne part* of the following: 1x10 =10

3 a. With a neat diagram discuss the three-level architecture of DBMS.

The three-level architecture, also known as the ANSI-SPARC architecture, is a widely accepted standard that defines the structure of a Database Management System (DBMS). Its primary objective is to separate the user's view of the data from the physical representation of the data. This separation is known as **data independence**. The architecture divides the DBMS into three distinct levels of abstraction:



1. External Level (View Level)

This is the highest level of database abstraction. It is the level closest to the users.

- **Description:** It describes how a specific group of users views the database. Each user group needs to see different subsets of the data. For instance, an employee view might show their own pay stub, while a manager's view might show their entire team's salary information but not personal medical details.
- **Purpose:** To provide a customized and simplified view of the database. Users see only the data relevant to them and are hidden from the complexities of the rest of the database structure. There can be multiple different external views.

2. Conceptual Level (Logical Level)

This is the middle level of abstraction. It acts as the intermediary between the external and internal levels.

- **Description:** This level describes *what* data is stored in the database and *what relationships* exist among those data points. It is often referred to as the community view of the entire database structure. This level is defined by the conceptual schema (typically designed using an Entity-Relationship (ER) model). It does not specify *how* the data is physically stored.
- **Purpose:** To create a unified, clear representation of the entire organization's data. It defines all the entities, attributes, relationships, security constraints, and semantic information. It is maintained by the Database Administrator (DBA).

3. Internal Level (Physical Level)

This is the lowest level of database abstraction.

- **Description:** It describes *how* the data is physically stored on the actual storage device (like hard disks or SSDs). It deals with low-level details, such as complex file structures, indexing methods, data compression, and hashing algorithms used to optimize retrieval speed.
- **Purpose:** To optimize the physical storage and retrieval of data. It ensures that the conceptual data model can be translated efficiently into the actual bits and bytes on the hardware.

3b. Explain with example, how data redundancy leads to data inconsistency.

Data Redundancy refers to the practice of keeping multiple copies of the same piece of information in different parts of a database.

Data Inconsistency is the direct and damaging consequence of uncontrolled redundancy. It occurs when different copies of the same data items, which should be identical, contain conflicting or different values.

Data redundancy is the **main cause** of data inconsistency. When multiple copies of the same data exist, any update must be applied to all copies. If even one copy is not updated, inconsistency arises.

Thus:**More redundancy ⇒ Higher chances of inconsistency**

Example: A Bank Customer Profile

Imagine a simple, inefficient bank database. The "Loans" department and the "Credit Cards" department operate separately and keep their own redundant tables of customer contact information.

Table 1: Loan_Department (Copy 1 of Customer Info)

CustomerID	Name	Address (Redundant)	City	Zip
C101	John Doe	**123 Maple St**	Chicago	60601
C102	Jane Smith	456 Oak Ave	New York	10001

Table 2: CreditCard_Department (Copy 2 of Customer Info)

CustomerID	Name	Address (Redundant)	City	Zip
C101	John Doe	**123 Maple St**	Chicago	60601
C103	Bob Jones	789 Pine Ln	Boston	02108

Suppose the customer "John Doe" (C101) decides to move. He calls the bank to update his address. He reaches a representative in the Loan Department. The representative updates only the table they use.

Transaction (Updating Loan Department only):

Table 1: Loan_Department (UPDATED)

CustomerID	Name	Address	City	Zip
C101	John	**999 Elm Blvd**	Chicago	60601
C102	Jane	456 Oak Ave	New	10001

Table 2: CreditCard_Department (NOT UPDATED)

CustomerID	Name	Address	City	Zip
C101	John	**123 Maple St**	Chicago	60601
C103	Bob	789 Pine Ln	Boston	02108

We now have **Data Inconsistency**. The single real-world fact (John Doe's current address) is represented by two conflicting pieces of data in the system.

- The Loans Department will mail his statements to **999 Elm Blvd** (Correct).
- The Credit Card Department will mail his statements to **123 Maple St** (Incorrect), where the new tenants might accidentally open his sensitive financial information.

---

4 Note: - Attempt any o*ne part* of the following: 1x10 =10

4 a. For a relation ClientMaster[Client No(Varchar, primary key), Name(Varchar, not null), Address1(Varchar), City(Varchar, default 'Delhi), Pincode(Number), Bal Due(Float), find the SQL queries for following-
1. Create the given relation.
2. Add new field as state in the relation
3. Remove the city field from the relation.
4. Remove the existence of ClientMaster relation
5. Rename clientMaster to Client Master

1. Create the Given Relation

```
CREATE TABLE ClientMaster (
   Client_No VARCHAR(20) PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Address1 VARCHAR(100),
   City VARCHAR(50) DEFAULT 'Delhi',
   Pincode NUMBER,
   Bal_Due FLOAT
);
```

2. Add New Field "State" in the Relation

```
ALTER TABLE ClientMaster ADD State VARCHAR(50);
```

3. Remove the City Field from the Relation

```
ALTER TABLE ClientMaster DROP COLUMN City;
```

4. Remove the Entire ClientMaster Relation

```
DROP TABLE ClientMaster;
```

5. Rename ClientMaster to "Client Master"

```
ALTER TABLE ClientMaster RENAME TO "Client Master";
```

---

4 b. A table named EMP(Empid, Name, DOB, Address, Passport_No, Lisence_No. , SSN) is there. Find out the following:
Alternative Keys, Non-key Attributes, Non-Prime attributes, Prime Attribute.

In a standard employment database for this table structure:

- **Empid**: Unique ID assigned by the company.
- **Passport_No**: Unique identification issued by the government.
- **Lisence_No**: Unique identification issued by the transport authority.
- **SSN**: Social Security Number (Unique per individual).

Primary Key (Assumed)

Usually, in a database like this, **Empid** is chosen as the Primary Key.

Candidate Keys

The set of all attributes that can uniquely identify a record:

- {Empid, Passport_No, Lisence_No, SSN}

1. Alternative Keys

Alternative Keys are all Candidate Keys that were **not** selected as the Primary Key.

- **Alternative Keys:** Passport_No, Lisence_No, SSN

2. Non-Key Attributes

While often used interchangeably with Non-Prime, in strict database theory, these are attributes that do not belong to the **Primary Key**.

- **Non-Key Attributes:** Name, DOB, Address, Passport_No, Lisence_No, SSN

3. Non-Prime Attributes

Attributes that are not part of any Candidate Key.

- **Non-Prime Attributes:** Name, DOB, Address

4. Prime Attributes

An attribute is "Prime" if it is a member of **any** Candidate Key.

- **Prime Attributes:** Empid, Passport_No, Lisence_No, SSN

---

5 a. Demonstrate inner-join and outer-join with suitable example.

In relational databases, a **Join** is a binary operation that allows you to combine related data from two or more tables based on a common column (the **Join Condition**).

Types of join:

Inner Join: The **Inner Join** is the most common type of join. It focuses on the **intersection** of two sets. It compares each row of Table A with each row of Table B. If the join condition evaluates to TRUE, the combined row is added to the result set.

Outer Join : Outer Joins are designed to handle **missing information**. They ensure that even if a record does not have a matching counterpart in the other table, it is not "lost" from the output.

A. Left Outer Join (Left Join)

- It preserves every row from the "Left" table (the one mentioned first in the SQL query).If a match is found in the right table, the values are joined. If no match is found, the result still contains the left row, but all columns from the right table are filled with NULL.

B. Right Outer Join (Right Join)

- The mirror image of the Left Join. It preserves every row from the "Right" table.All rows from the right table are included. If there is no match in the left table, the left-side columns are returned as NULL.

C. Full Outer Join

- This is a combination of both Left and Right Outer Joins. It represents the **Union** of both tables based on the join condition. It returns all records when there is a match in either the left or right table. It provides a complete view of all data in both tables, regardless of whether relationships exist.

Table: Students

StudentID	Name	Dept
101	Amit	CS
102	Priya	EC
103	Rahul	ME

Table: LibraryCards

CardID	StudentID	Status
501	101	Active
502	102	Active
504	105	Expired

2. Inner Join

An **Inner Join** returns only the rows where there is a match in **both** tables. If a row in the first table does not have a corresponding match in the second, it is omitted from the result.

SQL Example:
SELECT Students.Name, LibraryCards.CardID FROM Students INNER JOIN LibraryCards ON Students.StudentID = LibraryCards.StudentID;
Result:

Name	CardID
Amit	501
Priya	502

Note: Rahul is excluded because he has no CardID, and **Card 504** is excluded because Student 105 is not in our Students table.

3. Outer Join
Outer joins return matched rows **plus** the unmatched rows from one or both tables. The missing values are filled with NULL. There are three types:

A. Left Outer Join
Returns all rows from the **left** table (Students), and the matched rows from the right table.

Name	CardID
Amit	501
Priya	502
Rahul	**NULL**

B. Right Outer Join
Returns all rows from the **right** table (LibraryCards), and the matched rows from the left table.

Name	CardID
Amit	501
Priya	502
NULL	**504**

C. Full Outer Join
Returns all rows when there is a match in either the left or the right table.

Name	CardID
Amit	501
Priya	502
Rahul	**NULL**
NULL	**504**

---

5 b. How is a trigger different from procedure? Illustrate a trigger which will insert the updated salary as well as the old salary of an employee into a table named emp audit.
Trigger
A **trigger** is a special type of stored program in a database that is **automatically executed or "fired"** in response to certain events occurring on a table or view. These events are typically **INSERT, UPDATE, or DELETE** operations.
Triggers are primarily used to **enforce data integrity, maintain consistency, and implement automatic actions** without user intervention.
Uses of Trigger

- Enforcing business rules
- Maintaining audit logs
- Automatically updating related tables
- Preventing invalid data entry

Stored Procedure

A **stored procedure** is a **precompiled collection of SQL statements** stored in the database that performs a specific task. It is executed **explicitly by the user or application** when required.
Stored procedures are used to **encapsulate business logic, improve performance, and promote code reusability**.

Uses of Stored Procedure

- Performing complex database operations
- Implementing business logic
- Data validation and processing
- Reducing network traffic (multiple queries in one call)

A **trigger** is an automatic, event-driven program used mainly for maintaining integrity and enforcing rules, whereas a **stored procedure** is a user-invoked program used to perform specific tasks and implement business logic. Both play an important role in database management but differ significantly in execution and purpose.

```
CREATE TRIGGER salary_update_audit
AFTER UPDATE ON EMP
FOR EACH ROW
BEGIN
    INSERT INTO emp_audit (empid, old_salary, new_salary)
    VALUES (OLD.Empid, OLD.salary, NEW.salary);
END //
```

6a. what is transitive functional dependency? Explain how insertion anomaly, Updation anomaly and deletion anomaly occurs when a relation is having transitive functional dependency?

Transitive Dependency :A **Transitive Dependency** occurs in a database when an indirect relationship exists between attributes. Specifically, it happens when a non-key attribute depends on another non-key attribute, rather than depending directly on the Primary Key.
In formal terms, if you have three attributes A, B and C in a relation:

1. A → B (A determines B)
2. B → C (B determines C)
3. Therefore, A→ C (A transitively determines C)

For a table to be in **Third Normal Form (3NF)**, all such transitive dependencies must be removed.
Example: Imagine a **Student** table:

- **Attributes:** StudentID (Key), DeptName, DeptHOD
- **Dependencies:** StudentID →DeptName and DeptName →DeptHOD.
- **Transitive Dependency:** StudentID →DeptHOD (via DeptName).

StudentID	DeptName	DeptHOD
101	CS	Dr. Smith
102	CS	Dr. Smith
103	EE	Dr. Brown

1. Insertion Anomaly : An insertion anomaly occurs when you cannot record a fact because you are missing a piece of unrelated information.

- **How it happens:** Suppose the college opens a new "Biology" department with "Dr. Green" as the HOD.

- **The Problem:** Because the DeptHOD is transitively tied to the StudentID, you cannot add the new department to this table until a student actually enrolls in it. You are forced to leave the Primary Key (StudentID) null, which is not allowed (**Entity Integrity Constraint**).

2. Updation (Update) Anomaly: An update anomaly occurs when data is redundant, and changing it in one place but not others leads to inconsistency.

- **How it happens:** Suppose "Dr. Smith" retires and "Dr. Taylor" becomes the new HOD for the CS department.
- **The Problem:** You must search every single row where DeptName is "CS" and update the DeptHOD. If the table has 10,000 students and you miss just one row, the database will report two different HODs for the same department.

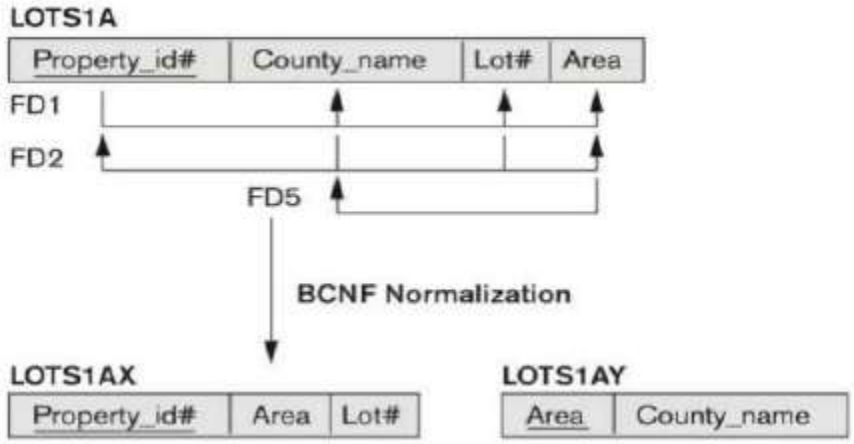
3. Deletion Anomaly: A deletion anomaly occurs when deleting one fact results in the unintentional loss of a completely different fact.

- **How it happens:** Imagine Student 103 is the only student currently enrolled in the "EE" department. If that student drops out and you delete their row.
- **The Problem:** You have not only lost the student's record but also the information that the "EE" department exists and that "Dr. Brown" is its HOD. That departmental data is gone forever.

6b. Define BCNF. Illustrate your answer with a suitable example.

It is an advance version of 3NF that's why it is also referred as 3.5NF. BCNF is stricter than 3NF. A table is in BCNF if

- It is in 3NF
- Every functional dependency X→Y, X should be the super key of the table.



In the above diagram there are two candidate keys property_id and country_name+lot (composite candidate key)and three functional Dependencies.
FD1 { property_id → country_name, lot, Area}
FD2 { country_name, lot → property_id,Area}
FD5 { Area → country_name }

 FD1,FD2 follows the Rule of BCNF.(ie Every functional dependency X→Y, X should be the super key of the table.)
 But FD5 does not follow this Rule ie (area is not a Super key.)

According to FD2 the Relation is in 3NF, but not in BCNF. Hence decompose this relation into two relations LOTS1AX and LOTS1AY.

7a. what do you mean by concurrency? Demonstrate how 2 phase locking protocol can be useful for controlling the concurrency. Discuss the different variations of 2PL.

In a multi-user database environment, it's common for numerous users to want to access and modify the database simultaneously. This is known as concurrent execution. Imagine a busy library where multiple librarians are updating book records simultaneously. Just as multiple librarians shouldn't try to update the same record simultaneously, database users shouldn't interfere with each other's operations.

Executing transactions concurrently offers many benefits, like improved system resource utilization and increased throughput. However, these simultaneous transactions mustn't interfere with each other. The ultimate goal is to ensure the database remains consistent and correct. For instance, if two people try to book the last seat on a flight at the exact moment, the system must ensure that only one person gets the seat.Concurrent execution can lead to various challenges and to address these challenges, the DBMS employs concurrency control techniques. They are like traffic rules. Just as traffic rules ensure vehicles don't collide, concurrency control ensures transactions don't conflict.

The Two-Phase Locking Protocol

One protocol that ensures serializability is the **two-phase locking protocol**. This protocol requires that each transaction issue lock and unlock requests in two phases:

1. **Growing phase**. A transaction may obtain locks, but may not release any lock.
2. **Shrinking phase**. A transaction may release locks, but may not obtain any new locks.

Initially, a transaction is in the growing phase. The transaction acquires locks as needed. Once the transaction releases a lock, it enters the shrinking phase, and it can issue no more lock requests.

```
T1:lock-X(B);
 read(B);
B := B-50;
write(B);
lock-X(A);
read(A);
A := A + 50;
write(A);
unlock(B);
unlock(A);
commit;
```

Different variations of 2PL

Use senerio Transaction A (T_A) wants to **Read** the balance of Account 1 and **Write** (update) the balance of Account 2.

1. Strict 2PL

- **The Process:** T_A can release the Shared lock on Account 1 as soon as it's done reading. However, it **must hold** the Exclusive lock on Account 2 until the very moment it hits "Commit."
- **The Result:** Because the updated Account 2 was never "unlocked" before the commit, no other transaction could have seen the uncommitted change. **No cascading rollbacks.**

2. Rigorous 2PL

- **The Process:** T_A holds **both** the Shared lock on Account 1 and the Exclusive lock on Account 2 until the very end.
- **The Result:** This is the "gold standard" for reliability. It's like T_A is in a private room with all its files and doesn't unlock the door for anyone until it's completely finished.

3. Conservative (Static) 2PL

- **The Process:** Before T_A even starts, it tells the system: "I need Account 1 and Account 2." If Account 2 is currently being used by someone else, T_A sits on its hands and doesn't take *any* locks. It only starts when it can grab both simultaneously.
- **The Result:** Since T_A never holds one lock while begging for another, it can never get stuck in a "you have mine, I have yours" loop. **Deadlocks are impossible.**

7b. what do you mean by deadlock prevention? Discuss the Wait-Die and Wound-Wait deadlock prevention algorithm.

Deadlock: A deadlock is a condition where two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as no task ever gets finished and is in waiting state forever.
For example: In the student table, transaction T1 holds a lock on some rows and needs to update some rows in the grade table. Simultaneously, transaction T2 holds locks on some rows in the grade table and needs to update the rows in the Student table held by Transaction T1.
Now, the main problem arises. Now Transaction T1 is waiting for T2 to release its lock and similarly, transaction T2 is waiting for T1 to release its lock. All activities come to a halt state and remain at a standstill. It will remain in a standstill until the DBMS detects the deadlock and aborts one of the transactions.

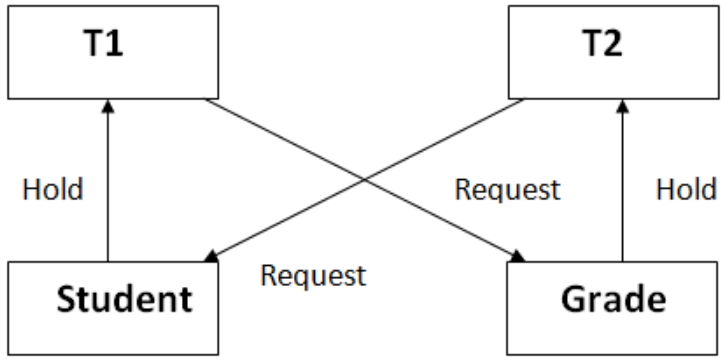


Figure: Deadlock in DBMS

<u>Deadlock Handling:</u> A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set.
For ex: There exists a set of waiting transactions T0, T1,... , Tn such that T0 is waiting for a data item that T1 holds, and T1 is waiting for a data item that T2 holds, and ... , and Tn−1 is waiting for a data item that Tn holds, and Tn is waiting for a data item that T0 holds. None of the transactions can make progress in such a situation.

There are two principal methods for dealing with the deadlock problem.

Deadlock Avoidance

- When a database is stuck in a deadlock state, then it is better to avoid the database rather than aborting or restating the database. This is a waste of time and resource.
- Deadlock avoidance mechanism is used to detect any deadlock situation in advance. A method like "wait for graph" is used for detecting the deadlock situation but this method is suitable only for the smaller database. For the larger database, deadlock prevention method can be used.

Deadlock Prevention

- Deadlock prevention method is suitable for a large database. If the resources are allocated in such a way that deadlock never occurs, then the deadlock can be prevented.
- The Database management system analyzes the operations of the transaction whether they can create a deadlock situation or not. If they do, then the DBMS never allowed that transaction to be executed.

Deadlock prevention mechanism proposes two schemes:

- **Wait-Die Scheme:** In this scheme, If a transaction requests a resource that is locked by another transaction, then the DBMS simply checks the timestamp to decide who has priority. The smaller the timestamp, the "older" the transaction.

  Suppose, there are two transactions T1 and T2, and Let the timestamp of any transaction T be TS (T). Checks if TS (T1) < TS (T2) –
  - if T1 is the older transaction and T2 has held some resource, then it allows T1 to wait until resource is available for execution. That means if a younger transaction has locked some resource and an older transaction is waiting for it, then an older transaction is allowed to wait for it till it is available.
  - If T1 is an older transaction and has held some resource with it and if T2 is waiting for it, then T2 is killed and restarted later with random delay but with the same timestamp. This scheme allows the older transaction to wait but kills the younger one.

Wound Wait Scheme: The **Wound-Wait** scheme is the "aggressive" counterpart to the Wait-Die scheme. It is also a timestamp-based **Deadlock Prevention** protocol, but it uses a different logic to decide who gets the resource. In this scheme, the older transaction doesn't just wait—it **wounds** the younger one.

- In this scheme, if an older transaction requests for a resource held by a younger transaction, then an older transaction forces a younger transaction to kill the transaction and release the resource. The younger transaction is restarted with a minute delay but with the same timestamp. If the younger transaction is requesting a resource that is held by an older one, then the younger transaction is asked to wait till the older one releases it.